

torch.qr#

<https://pytorch.org/docs/stable/generated/torch.qr.html>

Description:

Computes the QR decomposition of a matrix or a batch of matrices input, and returns a namedtuple (Q, R) of tensors such that $\text{input} = QR$ with Q being an orthogonal matrix or batch of orthogonal matrices and R being an upper triangular matrix or batch of upper triangular matrices. If some is True, then this function returns the thin (reduced) QR factorization. Otherwise, if some is False, this function returns the complete QR factorization. `torch.qr()` is deprecated in favor of `torch.linalg.qr()` and will be removed in a future PyTorch release. The boolean parameter `some` has been replaced with a string parameter `mode`. `Q, R = torch.qr(A)` should be replaced with

Function Signature

```
torch.qr(input: Tensor, some: bool = True, *, out: Union[Tensor, Tuple[Tensor, ...], List[Tensor], None])#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (tuple, optional) – tuple of Q and R tensors. The dimensions of Q and R are detailed in the description of some above.

Code Examples

```
Q, R = torch.linalg.qr(A)
```

```
Q, R = torch.linalg.qr(A, mode="complete")
```

```
>>> a = torch.tensor([[12., -51, 4], [6, 167, -68], [-4, 24, -41]])
>>> q, r = torch.qr(a)
>>> q
tensor([[ -0.8571,  0.3943,  0.3314],
        [ -0.4286, -0.9029, -0.0343],
        [ 0.2857, -0.1714,  0.9429]])
>>> r
tensor([[ -14.0000, -21.0000,  14.0000],
        [  0.0000, -175.0000,  70.0000],
        [  0.0000,   0.0000, -35.0000]])
>>> torch.mm(q, r).round()
tensor([[ 12., -51.,  4.],
        [  6., 167., -68.],
        [ -4.,  24., -41.]])
>>> torch.mm(q.t(), q).round()
tensor([[ 1.,  0.,  0.],
        [ 0.,  1., -0.],
        [ 0., -0.,  1.]])
>>> a = torch.randn(3, 4, 5)
>>> q, r = torch.qr(a, some=False)
>>> torch.allclose(torch.matmul(q, r), a)
True
>>> torch.allclose(torch.matmul(q.mT, q), torch.eye(5))
True
```

Notes & Warnings

⚠ WARNING

Warning `torch.qr()` is deprecated in favor of `torch.linalg.qr()` and will be removed in a future PyTorch release. The boolean parameter `some` has been replaced with a string parameter `mode`. `Q, R = torch.qr(A)` should be replaced with `Q, R = torch.linalg.qr(A)` `Q, R = torch.qr(A, some=False)` should be replaced with `Q, R = torch.linalg.qr(A, mode="complete")`

⚠ WARNING

Warning If you plan to backpropagate through QR, note that the current backward implementation is only well-defined when the first $\min(\text{input.size}(-1), \text{input.size}(-2))$ columns of input are linearly independent. This behavior will probably change once QR supports pivoting.

NOTE

Note This function uses LAPACK for CPU inputs and MAGMA for CUDA inputs, and may produce different (valid) decompositions on different device types or different platforms.

Detailed Documentation

Documentation

Computes the QR decomposition of a matrix or a batch of matrices input, and returns a namedtuple (Q, R) of tensors such that $\text{input} = QR$ with Q being an orthogonal matrix or batch of orthogonal matrices and R being an upper triangular matrix or batch of upper triangular matrices.

If some is True, then this function returns the thin (reduced) QR factorization. Otherwise, if some is False, this function returns the complete QR factorization.

`torch.qr()` is deprecated in favor of `torch.linalg.qr()` and will be removed in a future PyTorch release. The boolean parameter `some` has been replaced with a string parameter `mode`.

`Q, R = torch.qr(A)` should be replaced with

`Q, R = torch.qr(A, some=False)` should be replaced with

If you plan to backpropagate through QR, note that the current backward implementation is only well-defined when the first $\min(\text{input.size}(-1), \text{input.size}(-2))$ columns of input are linearly independent. This behavior will probably change once QR supports pivoting.

This function uses LAPACK for CPU inputs and MAGMA for CUDA inputs, and may produce different (valid) decompositions on different device types or different platforms.

`input` (Tensor) – the input tensor of size $(*, m, n)$ where $*$ is zero or more batch dimensions consisting of matrices of dimension $m \times n$.

`some` (bool, optional) –

Set to True for reduced QR decomposition and False for complete QR decomposition.

If $k = \min(m, n)$ then:

`some=True` : returns (Q, R) with dimensions (m, k), (k, n) (default)

`'some=False'`: returns (Q, R) with dimensions (m, m), (m, n)

out (tuple, optional) – tuple of Q and R tensors. The dimensions of Q and R are detailed in the description of some above.